

boba_extension_insight

Boba Browser Extension — Cross-Dimension Insights

Insight 1: The “Zero-Config Discovery” Pattern

Derived From: Dim01 (Boba Architecture) + Dim09 (API Integration) + Dim04 (Cross-Browser)

Insight: Boba’s multi-service architecture (Python FastAPI on 7187, Go on 7189, qBitTorrent on 8080) creates a discovery challenge for the extension. However, Boba’s `docker-compose.yml` and `.env.example` reveal that all services are co-located. The extension can implement **auto-discovery** by trying well-known ports on `localhost/127.0.0.1`, then fall back to manual configuration. This mirrors how Sonarr/Radarr auto-discover download clients.

Rationale: Most Boba users run all services on the same host. A UDP/TCP port scan of 7187, 7189, and 8080 on the local network can auto-detect the full stack within seconds. Combined with `/app/version` and `/api/v2/app/version` health endpoints, the extension can identify and configure itself automatically.

Implications: Reduces user onboarding from manual configuration (5+ fields) to a single “Auto-Discover” click. This is a significant UX advantage over existing qBitTorrent extensions.

Confidence: High

Insight 2: The “Unified Torrent Identity” Architecture

Derived From: Dim06 (Magnet Parsing) + Dim07 (Torrent Parsing) + Dim09 (API Integration)

Insight: Magnet links and .torrent files converge on a single identity: the BTIH (BitTorrent Info Hash). The extension can build a **unified torrent model** where every discovered item is normalized to `{infoHash, name, source, type: 'magnet'|'torrent-file'|'torrent-url'}`. This enables deduplication across an entire tab group, even when the same torrent appears as both a magnet link and a .torrent file.

Rationale: Both magnet links (via `xt=urn:btih:...`) and .torrent files (via SHA-1 of bencoded info dict) produce the same 40-char infohash. This is the canonical identity across all BitTorrent ecosystem tools.

Implications: Users never accidentally add the same torrent twice. The extension can show “already in queue” indicators. Boba itself already deduplicates by infohash, so the extension aligns perfectly with backend behavior.

Confidence: High

Insight 3: The “Progressive Enhancement” Content Strategy

Derived From: Dim08 (DOM Scraping) + Dim06 (Magnet Detection) + Dim10 (UI/UX)

Insight: Rather than trying to parse every possible torrent site, the extension should use a **3-tier detection strategy**: (1) Universal magnet/.torrent link scanning (works everywhere), (2) Site-specific selectors for top 20 torrent sites (optimized accuracy), (3) Text-based magnet detection for forums/Reddit. This progressive approach maximizes coverage without brittle site-specific maintenance.

Rationale: Dim08 research shows that `querySelectorAll('a[href^="magnet:"]')` catches 90% of magnets globally. The remaining 10% are text-based magnets on forums or dynamic SPAs. A hybrid approach with `MutationObserver` handles all cases.

Implications: The extension works on ANY website immediately, not just known torrent sites. This is unique — most existing extensions require site-specific support.

Confidence: High

Insight 4: The “Yandex Tab Group as Batch Job” Pattern

Derived From: Dim05 (Tab Groups) + Dim01 (Boba Architecture) + Dim10 (UI/UX)

Insight: Yandex Browser’s tab groups feature, combined with Boba’s batch torrent API, enables a powerful workflow: user organizes torrent search results into a tab group → right-clicks group → “Send All to Boba” → extension extracts ALL tabs’ URLs, parses each page for torrents, and submits as a single batch. This turns tab groups into **curated download queues**.

Rationale: `chrome.tabGroups.query()` + `chrome.tabs.query({groupId})` gives all URLs in a group. Boba’s `/api/v2/torrents/add` accepts multiple URLs separated by newlines. The batch flow is technically straightforward but creates a uniquely powerful user experience.

Implications: This feature alone differentiates the extension from all competitors. No existing extension treats tab groups as batch operations. It rewards users who organize their browsing.

Confidence: High

Insight 5: The “Extension as Boba Satellite” Architecture

Derived From: Dim01 (Boba Architecture) + Dim09 (API Integration) + Dim11 (Security)

Insight: The extension should be designed as a **satellite client** of Boba rather than a standalone tool. This means: (1) It uses Boba’s auth model (API keys, not its own), (2) It respects Boba’s configuration (categories, save paths), (3) It extends Boba’s UI rather than duplicating it. The popup shows “what’s on this page” while Boba’s dashboard shows “what’s downloading.”

Rationale: Boba already has a sophisticated Angular dashboard with real-time monitoring, search, and management. The extension’s role is to bridge the browser-to-Boba gap, not replace Boba’s UI. This avoids feature duplication and security complexity.

Implications: Simpler extension code (~30% less), single source of truth for configuration, and users get a consistent experience. The extension is a “send to” tool, not a management tool.

Confidence: High

Insight 6: The “Offline-Aware Queue” Resilience Pattern

Derived From: Dim09 (API Integration) + Dim03 (Extension Architecture) + Dim11 (Security)

Insight: Since Boba is self-hosted, it’s frequently offline (laptop closed, server rebooting, network changes). The extension must implement an **offline queue** with `chrome.storage.local` persistence: detect offline → queue torrents → retry with exponential backoff → sync when Boba is back. This is critical for self-hosted services but ignored by most extensions.

Rationale: `navigator.onLine` is unreliable for LAN services. A proactive health check (`/app/version` ping every 30s via `chrome.alarms`) combined with a persistent queue provides reliable delivery.

Implications: Users never lose a torrent because Boba was temporarily unreachable. The extension feels reliable even on unstable networks. The queue can be visualized in the popup for transparency.

Confidence: High

Insight 7: The “Privacy-First Permission Model”

Derived From: Dim11 (Security) + Dim03 (Architecture) + Dim04 (Cross-Browser)

Insight: Most torrent extensions request `<all_urls>` or broad host permissions, creating privacy risks. This extension can use **activeTab** as the primary permission with **optional** host permissions for popular torrent sites. Content scripts are injected via `chrome.scripting.executeScript()` on demand rather than at page load. This follows the principle of least privilege.

Rationale: `activeTab` grants temporary access only when the user clicks the extension icon. Combined with programmatic injection, the extension doesn’t see any page content until explicitly activated. This is approved by all extension stores more easily.

Implications: Better store approval rates, better user trust, no background page constantly scanning pages. The tradeoff is requiring a click before detecting torrents — acceptable for a “send to” tool.

Confidence: Medium (some store reviewers prefer declared `host_permissions`)

Insight 8: The “Multi-Client Gateway” Potential

Derived From: Dim02 (qBitTorrent API) + Dim09 (Integration) + Dim01 (Boba Architecture)

Insight: While the immediate target is Boba (which wraps qBitTorrent), the API integration layer can be abstracted to support **multiple torrent clients:** qBitTorrent native, Transmission (RPC), Deluge (WebUI), and rTorrent. The extension's architecture should have a `TorrentClient` interface with pluggable implementations.

Rationale: The qBitTorrent, Transmission, and Deluge APIs all share similar concepts: auth, add torrent (URL/file), list torrents, pause/resume. An adapter pattern abstracts these differences. The magnet-linker-browser-extension on GitHub already does this for Transmission.

Implications: Future-proof architecture. The Boba-specific implementation is just one adapter. Users without Boba can connect directly to their torrent client. This dramatically expands the addressable user base.

Confidence: Medium (adds complexity; should be v2 feature)

Insight 9: The “Real-Time Badge Sync” Opportunity

Derived From: Dim03 (Architecture) + Dim09 (API Integration) + Dim10 (UI/UX)

Insight: By polling `/sync/maindata` with incremental RID from qBitTorrent WebUI, the extension can show **live download progress** on its badge. The badge text shows download count, color shows status (green = downloading, blue = complete, red = error). This creates ambient awareness without opening any UI.

Rationale: `chrome.action.setBadgeText()` and `setBadgeBackgroundColor()` update instantly. `/sync/maindata` with RID gives delta updates (only changed torrents), making polling efficient even at 5-second intervals.

Implications: Users always see download status at a glance. No notification spam needed. The extension feels “alive” and connected to Boba.

Confidence: High

Insight 10: The “Enterprise-Grade Testing” Differentiator

Derived From: Dim12 (Testing) + Dim01 (Boba Quality) + Dim11 (Security)

Insight: Boba itself has enterprise-grade testing (pytest, Playwright, SonarQube, Snyk, Semgrep, Trivy). The extension should match this quality bar: 80%+ code coverage, automated E2E tests with real Boba instance in Docker, security scanning in CI, and signed releases. Most browser extensions have zero automated tests — this quality level is a differentiator.

Rationale: Boba’s `.github/workflows/`, `docker-compose.quality.yml`, and scanner configs (Sonar, Snyk, Semgrep, Trivy, Gitleaks) set a clear quality standard. The extension can mirror this with `jest --coverage`, `playwright test`, and the same security scanners.

Implications: Professional-grade reliability, easier contributions, confident releases. The CI/CD pipeline auto-publishes to all 5 browser stores on release.

Confidence: High